Convenciones utilizadas

Contenido teórico

Inicio de una nueva sección

Ejemplos y ejercicios prácticos

Soluciones de ejemplos y ejercicios

Material optativo

En esta clase

Vamos a conectar cuatro niveles de descripción:

$$\text{Operadores y matrices} \longrightarrow \text{Circuitos con SpinQit} \longrightarrow \text{Fase e interferencia} \longrightarrow \text{Hardware real}$$

> La idea central será comparar continuamente la predicción matemática con la ejecución del circuito.

Operadores

¿Qué hace un operador cuántico?

Un **operador cuántico** sobre un espacio de Hilbert es una **transformación lineal**

$$\hat{A} : \mathcal{H} \longrightarrow \mathcal{H}, \qquad |\psi\rangle \longmapsto \hat{A}|\psi\rangle.$$

Por ser lineal:

$$\hat{A}(c|\psi\rangle + d|\phi\rangle) = c\hat{A}|\psi\rangle + d\hat{A}|\phi\rangle.$$

Convención

- $\hat{A}$: operador abstracto sobre $\mathcal{H}$;
- A: matriz que representa a $\hat{A}$ en una base elegida.

Recordatorio de la clase 2

Una transformación lineal queda completamente determinada por su acción sobre los vectores de una base.

Si

$$v = c_1 e_1 + \cdots + c_n e_n,$$

entonces

$$T(v) = c_1 T(e_1) + \cdots + c_n T(e_n).$$

> Por eso, para conocer T alcanza con conocer $T(e_1), \ldots, T(e_n)$.

La matriz se construye con las imágenes de la base

Si $\mathcal{B} = (e_1, \ldots, e_n)$, la matriz que representa a T se construye colocando como columnas las coordenadas de:

$$T(e_1),\ T(e_2), \ldots, T(e_n).$$

Esquemáticamente:

$$[T]_{\mathcal{B}} = \begin{pmatrix} | & | & & | \\ [T(e_1)]_{\mathcal{B}} & [T(e_2)]_{\mathcal{B}} & \cdots & [T(e_n)]_{\mathcal{B}} \\ | & | & & | \end{pmatrix}.$$

Notación: $\mathcal{B} = (e_1, \ldots, e_n)$ indica una **base ordenada**. El orden determina las coordenadas y el orden de las columnas de $[T]_{\mathcal{B}}$. Es la misma regla del álgebra lineal.

La misma regla para un qubit

Para un qubit usamos la base computacional

$$\mathcal{B} = (|0\rangle, |1\rangle).$$

Supongamos:

$$\hat{A}|0\rangle = \begin{pmatrix} a \\ c \end{pmatrix}, \qquad \hat{A}|1\rangle = \begin{pmatrix} b \\ d \end{pmatrix}.$$

Colocando ambas imágenes como columnas:

$$A = \begin{pmatrix} \hat{A}|0\rangle & \hat{A}|1\rangle \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

> Cada columna nos dice qué hace el operador sobre un vector de la base.

Aplicar la matriz a un estado

Sea

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle.$$

Por linealidad:

$$\hat{A}|\psi\rangle = \alpha\hat{A}|0\rangle + \beta\hat{A}|1\rangle.$$

En coordenadas:

$$A\begin{pmatrix}\alpha\\ \beta\end{pmatrix} = \begin{pmatrix}a & b\\ c & d\end{pmatrix}\begin{pmatrix}\alpha\\ \beta\end{pmatrix} = \begin{pmatrix}a\alpha + b\beta\\ c\alpha + d\beta\end{pmatrix}.$$

> La multiplicación matricial implementa la acción del operador.

Algunos operadores importantes

Antes de escribir sus matrices, observemos qué hacen algunos operadores importantes:

$$
\begin{aligned}
\text{Identidad} &: \quad \hat{I}|0\rangle = |0\rangle, \qquad \hat{I}|1\rangle = |1\rangle \\
\text{Pauli-X} &: \quad \hat{X}|0\rangle = |1\rangle, \qquad \hat{X}|1\rangle = |0\rangle \\
\text{Pauli-Y} &: \quad \hat{Y}|0\rangle = i|1\rangle, \qquad \hat{Y}|1\rangle = -i|0\rangle \\
\text{Pauli-Z} &: \quad \hat{Z}|0\rangle = |0\rangle, \qquad \hat{Z}|1\rangle = -|1\rangle \\
\text{Hadamard} &: \quad \hat{H}|0\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}, \qquad \hat{H}|1\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}
\end{aligned}
$$

Sus matrices se obtienen colocando estas imágenes como columnas.

Ejemplo: construir la matriz identidad

Sabemos que

$$\hat{I}|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \hat{I}|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Por lo tanto:

$$I = \begin{pmatrix} \hat{I}|0\rangle & \hat{I}|1\rangle \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

> La matriz no se adivina: se construye a partir de la acción sobre la base.

Ejercicio: construir las matrices

A partir de su acción sobre $|0\rangle$ y $|1\rangle$, construir las matrices de:

1. Pauli-X;
2. Pauli-Y;
3. Pauli-Z;
4. Hadamard H.

> Para cada operador, escribir las dos imágenes en coordenadas y colocarlas como primera y segunda columna.

Solución: Pauli-X y Pauli-Y

Para X:

$$\hat{X}|0\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad \hat{X}|1\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Para Y:

$$\hat{Y}|0\rangle = \begin{pmatrix} 0 \\ i \end{pmatrix}, \qquad \hat{Y}|1\rangle = \begin{pmatrix} -i \\ 0 \end{pmatrix},$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

Solución: Pauli-Z y Hadamard

Para Z:

$$\hat{Z}|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \hat{Z}|1\rangle = \begin{pmatrix} 0 \\ -1 \end{pmatrix},$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Para H:

$$\hat{H}|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \hat{H}|1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Interpretar las matrices construidas

- $\hat{X}$ intercambia $|0\rangle$ y $|1\rangle$: es el **NOT cuántico**.
- $\hat{Y}$ también los intercambia, incorporando fases i y $-i$.
- $\hat{Z}$ conserva $|0\rangle$ y cambia el signo de $|1\rangle$.
- $\hat{H}$ transforma los estados de la base en superposiciones.

> La matriz es una representación compacta de la acción del operador.

Ejercicios de consolidación (trabajo autónomo): aplicar operadores

Sea un estado general de un qubit: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$.

Aplicar, mediante multiplicación matricial, los operadores:

1. Pauli-X;
2. Pauli-Y;
3. Pauli-Z;
4. Hadamard H.

Para cada caso:

1. escribir la matriz del operador;
2. calcular $A|\psi\rangle$;
3. expresar el resultado nuevamente en notación de Dirac.

Solución: Pauli-X y Pauli-Y

Para Pauli-X:

$$X|\psi\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \beta \\ \alpha \end{pmatrix}.$$

Por lo tanto:

$$\boxed{\hat{X}|\psi\rangle = \beta|0\rangle + \alpha|1\rangle}.$$

Para Pauli-Y:

$$Y|\psi\rangle = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} -i\beta \\ i\alpha \end{pmatrix}.$$

Por lo tanto:

$$\boxed{\hat{Y}|\psi\rangle = -i\beta|0\rangle + i\alpha|1\rangle}.$$

Solución: Pauli-Z y Hadamard

Para Pauli-Z:

$$Z|\psi\rangle = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \alpha \\ -\beta \end{pmatrix}.$$

Por lo tanto:

$$\boxed{\hat{Z}|\psi\rangle = \alpha|0\rangle - \beta|1\rangle}.$$

Para Hadamard:

$$H|\psi\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} \alpha + \beta \\ \alpha - \beta \end{pmatrix}.$$

Por lo tanto:

$$\boxed{\hat{H}|\psi\rangle = \frac{\alpha + \beta}{\sqrt{2}}|0\rangle + \frac{\alpha - \beta}{\sqrt{2}}|1\rangle}.$$

¿Todo operador lineal puede ser una compuerta?

No. No todo operador lineal puede representar una **compuerta cuántica**. Una compuerta cuántica debe transformar estados normalizados en estados normalizados:

$$\|\hat{U}|\psi\rangle\| = 1 \qquad \text{si} \qquad \||\psi\rangle\| = 1.$$

Ahora,

$$\|\hat{U}|\psi\rangle\|^2 = \langle \hat{U}\psi|\hat{U}\psi\rangle = \langle\psi|\hat{U}^\dagger\hat{U}|\psi\rangle.$$

> Para que la norma se conserve para todo $|\psi\rangle$, exigimos
>
> $$\hat{U}^\dagger\hat{U} = \hat{I}.$$

Operadores unitarios

Un operador $\hat{U}$ es **unitario** si $\hat{U}^\dagger \hat{U} = \hat{U}\hat{U}^\dagger = \hat{I}$.

Por lo tanto, $\hat{U}^{-1} = \hat{U}^\dagger$.

Además, para cualesquiera $|\phi\rangle$ y $|\psi\rangle$: $\langle \hat{U}\phi | \hat{U}\psi \rangle = \langle \phi | \psi \rangle$.

Así se conservan:

- la **norma** de los estados;
- sus **productos internos y solapamientos**;
- la **reversibilidad** de la evolución.

> En una base ortonormal, un operador unitario $\hat{U}$ se representa mediante una matriz unitaria U, que satisface $U^\dagger U = UU^\dagger = I$.

Un operador que no sirve como compuerta

Consideremos

$$D = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}.$$

Entonces:

$$D^\dagger D = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix} \neq I.$$

Podemos ver con un ejemplo que no conserva la norma:

$$\hat{D}|1\rangle = 2|1\rangle, \qquad \|\hat{D}|1\rangle\| = 2.$$

> $\hat{D}$ es lineal, pero no puede representar una compuerta cuántica porque no conserva la norma.

Ejercicio: ¿Es una compuerta cuántica válida?

Consideremos un operador lineal $\hat{A}$ definido por su acción sobre la base computacional:

$$\hat{A}|0\rangle = |+\rangle, \qquad \hat{A}|1\rangle = |1\rangle.$$

Los dos estados resultantes están normalizados:

$$\||+\rangle\| = \||1\rangle\| = 1.$$

¿Es suficiente para concluir que $\hat{A}$ es unitario?

1. Calcula el producto interno entre los estados de entrada:

$$\langle 0|1\rangle.$$

2. Calcula el producto interno entre los estados transformados:

$$\langle \hat{A}0|\hat{A}1\rangle = \langle +|1\rangle.$$

3. Compara ambos resultados y decide si $\hat{A}$ puede representar una compuerta cuántica.

Solución: no alcanza con normalizar

Antes de aplicar el operador:

$$\langle 0|1\rangle = 0.$$

Después de aplicar el mismo operador a ambos estados:

$$\langle \hat{A}0|\hat{A}1\rangle = \langle +|1\rangle = \frac{1}{\sqrt{2}}(\langle 0| + \langle 1|)|1\rangle = \frac{1}{\sqrt{2}}.$$

Por lo tanto,

$$\langle \hat{A}0|\hat{A}1\rangle \neq \langle 0|1\rangle.$$

El operador $\hat{A}$ **no preserva el producto interno** y, por lo tanto, no es unitario.

> No alcanza con comprobar que algunos estados individuales siguen normalizados. Una transformación unitaria debe preservar también las relaciones geométricas entre los estados, como su ortogonalidad.

Ejercicio: comprobar una compuerta

Verificar que la matriz de Hadamard

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

es unitaria, es decir:

$$H^\dagger H = I.$$

> Observar primero que H es real y simétrica.

Solución: Hadamard es unitaria

Como H es real y simétrica:

$$H^\dagger = H.$$

Entonces:

$$H^\dagger H = H^2 = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I.$$

Por lo tanto, H es **unitaria**.

Además:

$$\hat{H}^2 = I.$$

Es decir, aplicar Hadamard dos veces devuelve cualquier estado a su estado inicial.

El orden de las operaciones

Si primero aplicamos $\hat{U}_1$ y después $\hat{U}_2$:

$$|\psi_{\text{final}}\rangle = \hat{U}_2\hat{U}_1|\psi_{\text{inicial}}\rangle.$$

> La operación que se aplica primero aparece más a la derecha.

En general:

$$\hat{U}_2\hat{U}_1 \neq \hat{U}_1\hat{U}_2.$$

Primeros pasos con SpinQit

¿Qué es SpinQit?

SpinQit es el framework que utilizaremos para:

- describir circuitos cuánticos desde Python;
- compilar esos circuitos a una representación ejecutable;
- dibujar el circuito;
- probarlo en un simulador local;
- enviarlo, cuando corresponda, a la SpinQ Triangulum.

La misma descripción de alto nivel nos permite comparar un resultado ideal (usando un simulador) con una ejecución física (usando la máquina real).

Preparar el entorno de trabajo

La documentación de SpinQit indica:

- SpinQit requiere **Python 3.8, 3.9 o 3.10**;
- pruebas documentadas con **Python 3.8.13, 3.9.12 y 3.10.20**;
- uso recomendado de un entorno Anaconda o equivalente;
- instalación mediante `pip install spinqit`.

Siempre que sea posible, conviene trabajar en un **entorno virtual dedicado** para evitar conflictos con otras bibliotecas o versiones de Python.

Nuestro Primer Circuito Cuántico

Paso 1: importaciones

Para nuestros primeros ejemplos importaremos:

```
from spinqit import Circuit, H, X, Y, Z, draw
from spinqit import get_basic_simulator, get_compiler
from spinqit import BasicSimulatorConfig
```

Estas herramientas nos permiten:

- construir circuitos, aplicar compuertas de un qubit, compilarlos, dibujarlos y ejecutarlos con el simulador local.

Paso 2: crear el circuito

A continuación agregar estas dos instrucciones

```
circ = Circuit()
q = circ.allocateQubits(1)
```

La variable `circ` representa el circuito.

La instrucción `allocateQubits(1)` reserva exactamente un qubit. La expresión `q[0]` identifica ese qubit dentro del circuito.

`q[0]` no es el vector de estado: es una referencia que utilizamos para indicar dónde aplicar una compuerta.

El estado inicial

En el modelo de circuito, un qubit recién asignado comienza en

$$|0\rangle.$$

No necesitamos escribir una instrucción para preparar ese estado.

Por eso, cuando agregamos la compuerta X, la evolución esperada es

$$|0\rangle \xrightarrow{\hat{X}} |1\rangle.$$

Paso 3: agregar Pauli-X

Para aplicar X:

Agregar la siguiente instrucción

```
circ << (X, q[0])
```

El operador `<<` agrega una instrucción al circuito.

La forma general es:

```
circ << (COMPUERTA, q[0])
```

Todavía no estamos ejecutando la compuerta: estamos construyendo la descripción del circuito.

Paso 4: compilar

Para que un backend pueda ejecutar el circuito, debemos compilarlo:

```
compiler = get_compiler("native")
ir = compiler.compile(circ, 0)
```

La variable `ir` contiene una representación intermedia del circuito.

El segundo argumento indica el nivel de optimización utilizado en estos ejemplos.

El nivel cero indica que no estamos haciendo ninguna optimización.

Paso 5: dibujar el circuito

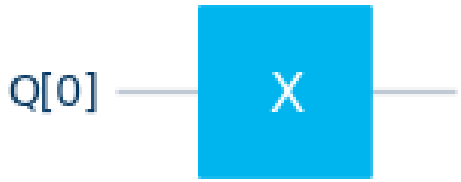
Una vez compilado, podemos generar una imagen del circuito:

Agregar la siguiente instrucción

```
draw(ir, "circuitoX.png")
```

La función `draw` recibe el circuito compilado y el nombre del archivo de salida.

El archivo `circuitoX.png` debería mostrar lo siguiente:



Paso 6: configurar el simulador

El backend local se obtiene con `get_basic_simulator()`:

```
engine = get_basic_simulator()
config = BasicSimulatorConfig()
config.configure_shots(1024)
config.configure_measure_qubits([0])
```

- `engine` representa el simulador;
- `config` contiene la configuración;
- 1024 es la cantidad de shots solicitada;
- `configure_measure_qubits([0])` indica que observaremos el qubit reservado.

Paso 7: ejecutar

La ejecución utiliza el circuito compilado `ir`:

```
result = engine.execute(ir, config)

print("counts:", result.counts)
print("probabilities:", result.probabilities)
```

El objeto `result` contiene la información que devuelve el backend.

El flujo completo puede resumirse como

$$\text{Circuit} \longrightarrow \text{compiler} \longrightarrow \text{ir} \longrightarrow \text{engine} \longrightarrow \text{result}.$$

Analizar el resultado de X

La predicción matemática es

$$\hat{X}|0\rangle = |1\rangle.$$

En el simulador ideal, con 1024 shots, obtenemos:

```
counts: {'1': 1024}
probabilities: {'1': 1.0}
```

Esto significa que todos los resultados fueron 1 y, por lo tanto,

$$P(1) = \frac{\text{cantidad de resultados } 1}{\text{cantidad total de shots}} = \frac{1024}{1024} = 1.$$

> La predicción matemática y la ejecución del circuito coinciden: el resultado 1 aparece con probabilidad uno.

Superposición con Hadamard

Reemplazar X por H

Para preparar una superposición, cambiamos la compuerta:

```
...

circ << (H, q[0])

...

draw(ir, "circuitoH.png")

...
```

Abrir `circuitoH.png` permite verificar que la compuerta dibujada es H.

Analizar los resultados de H

La predicción matemática es

$$\hat{H}|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

Por lo tanto,

$$P(0) = \frac{1}{2}, \qquad P(1) = \frac{1}{2}.$$

En el simulador ideal:

```
counts: {'0': 512, '1': 512}
probabilities: {'0': 0.5, '1': 0.5}
```

Acabamos de crear nuestra primera superposición mediante código.

Inspeccionar el vector de estado

El simulador puede mostrar el estado final:

```
print("states:", result.states)
```

Para $\hat{H}|0\rangle$ esperamos algo equivalente a

```
states: [(0.7071067811865475+0j), (0.7071067811865475+0j)]
```

Las dos entradas son amplitudes complejas:

$$\alpha = \frac{1}{\sqrt{2}}, \qquad \beta = \frac{1}{\sqrt{2}}.$$

De amplitudes a probabilidades

La regla de Born da:

$$P(0) = |\alpha|^2 = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2},$$

$$P(1) = |\beta|^2 = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}.$$

El vector de estado contiene más información que la distribución de probabilidades: también conserva las fases de las amplitudes.

Simulador y hardware

Inspeccionar `states` es posible porque el simulador mantiene en memoria el vector de estado ideal.

En un dispositivo físico:

- no podemos leer directamente las amplitudes;
- una medición produce un resultado clásico;
- necesitamos repetir el circuito y estudiar la estadística;
- las probabilidades pueden desviarse de las ideales por ruido.

Esta diferencia será importante al utilizar la Triangulum.

La esfera de Bloch y la fase

Visualizar el estado de un qubit

La representación como vector columna es precisa, pero no siempre es fácil de imaginar.

Para un estado puro de un qubit podemos utilizar la **esfera de Bloch**:

- el polo norte representa $|0\rangle$;
- el polo sur representa $|1\rangle$;
- el ecuador contiene estados con probabilidades iguales de medir 0 y 1.

> Cada punto de la esfera representa un estado físico puro, es decir, un rayo del espacio de Hilbert.

Estados sobre la esfera

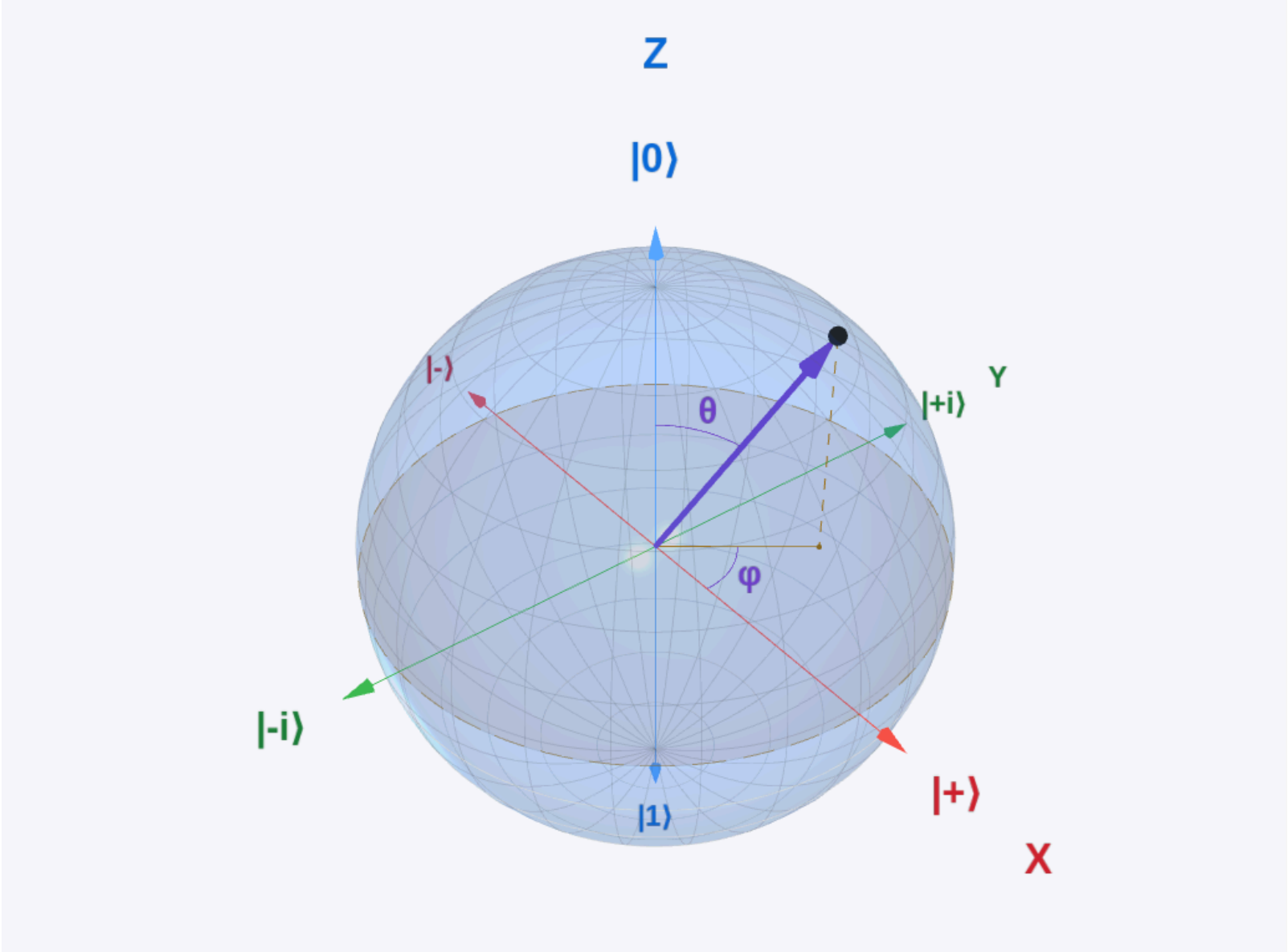
Salvo una fase global, todo estado puro de un qubit puede escribirse como

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\theta}{2}\right)|1\rangle.$$

- θ determina la posición entre los polos;
- φ determina la posición alrededor del eje Z y representa la **fase relativa**.
- Con $0 \leq \theta \leq \pi$ y $0 \leq \varphi < 2\pi$.

La fase global no aparece en la esfera de Bloch porque no modifica el estado físico.

Dos estados puros ortogonales se representan mediante **puntos antipodales** de la esfera de Bloch.



Probar la aplicación interactiva AppEsferaDeBloch.html

Compuertas como rotaciones

En la esfera de Bloch, algunas compuertas pueden interpretarse geométricamente como rotaciones:

- $\hat{X}$: rotación de 180° alrededor del eje X;
- $\hat{Y}$: rotación de 180° alrededor del eje Y;
- $\hat{Z}$: rotación de 180° alrededor del eje Z;
- $\hat{H}$: rotación de 180° alrededor de un eje a 45° entre X y Z.

> Estas equivalencias son sobre la esfera de Bloch: posibles diferencias por una fase global no modifican el estado físico.

La fase con la compuerta Z

Consideremos aplicar Z al estado $|+\rangle$

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{\hat{z}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

Las probabilidades en la base computacional no cambian:

$$P(0) = P(1) = \frac{1}{2}.$$

Sin embargo, el estado sí cambió: la fase relativa pasó de 0 a π. Son estados distintos, incluso son incluso ortogonales: $\langle +|-\rangle = 0$.

> Una medición en una base determinada puede no revelar una diferencia de fase relativa. Para hacerla observable debemos convertir esa diferencia de fase en una diferencia de probabilidades.

Interferencia

Aplicar $\hat{H}$ dos veces

La identidad matemática entre operadores es

$$\hat{H}^2 = \hat{I}.$$

Por lo tanto,

$$\hat{H}\left(\hat{H}|0\rangle\right) = |0\rangle.$$

El primer $\hat{H}$ crea la superposición y el segundo $\hat{H}$ recombina sus amplitudes, recuperando $|0\rangle$ y deshaciendo la primera transformación.

Circuito `HH`

Agregamos otra compuerta `H` y graficamos en "`circuitoHH.png`"

```
...

circ = Circuit()
q = circ.allocateQubits(1)
circ << (H, q[0])
circ << (H, q[0])

...

draw(ir, "circuitoHH.png")

...
```

Analizar Circuito `HH`



La salida es:

```
counts: {'0': 1024}
probabilities: {'0': 1.0}
states: [(1+0j), 0j]
```

El estado final volvió a ser $|0\rangle$.

La explicación matricial y la ejecución del circuito coinciden:

$$\hat{H}^2|0\rangle = |0\rangle.$$

Interferencia constructiva y destructiva

Después del primer Hadamard:

$$|0\rangle \xrightarrow{\hat{H}} |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

Aplicamos ahora el segundo Hadamard:

$$\begin{aligned}
\hat{H}|+\rangle &= \hat{H}\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \\
&= \frac{1}{\sqrt{2}}\left(\hat{H}|0\rangle + \hat{H}|1\rangle\right) \\
&= \frac{1}{2}\left[(|0\rangle + |1\rangle) + (|0\rangle - |1\rangle)\right] \\
&= \frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle + \frac{1}{2}|0\rangle - \frac{1}{2}|1\rangle \\
&= |0\rangle.
\end{aligned}$$

Interferencia constructiva y destructiva

En el estado final:

$$|0\rangle = 1|0\rangle + 0|1\rangle.$$

Las amplitudes se combinaron así:

$$\text{para } |0\rangle : \qquad \frac{1}{2} + \frac{1}{2} = 1$$

$$\text{para } |1\rangle : \qquad \frac{1}{2} - \frac{1}{2} = 0$$

- en $|0\rangle$ hubo **interferencia constructiva**;
- en $|1\rangle$ hubo **interferencia destructiva**.

> La interferencia cuántica consiste en el **refuerzo** o **debilitamiento** de amplitudes al combinarse sus contribuciones. En algunos casos, el debilitamiento puede llegar a una **cancelación total**.

Cambiar la última compuerta H por la compuerta Z

Ahora construiremos el circuito `HZ`:

```
...

circ = Circuit()
q = circ.allocateQubits(1)
circ << (H, q[0])
circ << (Z, q[0])

...

draw(ir, "circuitoHZ.png")

...
```

Analizar el Circuito `HZ` (operador $\hat{Z}\hat{H}$)

Si el circuito aplica primero $\hat{H}$ y luego $\hat{Z}$:

$$\hat{Z}\hat{H}|0\rangle = \hat{Z}\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle.$$

El vector de estado es

$$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}.$$

Al medir en la base computacional:

$$P(0) = P(1) = \frac{1}{2}.$$

> La compuerta Z cambió la **fase relativa**, pero esta medición no revela ese cambio.

El sándwich $\hat{H}\hat{Z}\hat{H}$

Agregamos una última compuerta H:

```
...

circ = Circuit()
q = circ.allocateQubits(1)
circ << (H, q[0])
circ << (Z, q[0])
circ << (H, q[0])

...

draw(ir, "circuitoHZH.png")

...
```

Revelar la fase mediante interferencia

La salida ideal de $\hat{H}\hat{Z}\hat{H}$ es:

```
counts: {'1': 1024}
probabilities: {'1': 1.0}
states: [0j, (1+0j)]
```

La secuencia transforma el estado inicial de la siguiente manera:

$$|0\rangle \xrightarrow{\hat{H}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{\hat{Z}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \xrightarrow{\hat{H}} |1\rangle.$$

La diferencia de fase relativa, que no era visible al medir $\hat{Z}\hat{H}|0\rangle$ en la base computacional, se convirtió ahora en una diferencia observable.

¿Será $\hat{H}\hat{Z}\hat{H} = \hat{X}$?

El circuito anterior produjo

$$\hat{H}\hat{Z}\hat{H}|0\rangle = |1\rangle.$$

Pero también sabemos que

$$\hat{X}|0\rangle = |1\rangle.$$

Por lo tanto, ambos operadores actúan igual sobre $|0\rangle$.

> ¿Podemos concluir que $\hat{H}\hat{Z}\hat{H} = \hat{X}$? Para demostrarlo debemos comparar los operadores, no solamente su acción sobre un estado.

Ejercicios de consolidación (trabajo autónomo)

Utilizando las matrices

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

demostrar mediante multiplicación matricial que:

$$\boxed{HZH = X}$$

y que

$$\boxed{HXH = Z}.$$

Solución: demostrar que $HZH = X$

Primero calculamos:

$$ZH = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}.$$

Luego:

$$\begin{aligned} HZH &= \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = X. \end{aligned}$$

Por lo tanto:

$$\boxed{HZH = X}.$$

Solución: demostrar que $HXH = Z$

Primero calculamos:

$$XH = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}.$$

Luego:

$$\begin{aligned} HXH &= \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = Z. \end{aligned}$$

Por lo tanto:

$$\boxed{HXH = Z}.$$

La compuerta Hadamard intercambia los operadores X y Z:
$HZH = X$ y $HXH = Z$.

Usar AppEsferaDeBloch.html para verificar el efecto de aplicar H, HH, HZ y HZH

Ejercicios de consolidación (trabajo autónomo): encontrar el error

Se desea ejecutar un circuito para evaluar $\hat{H}|0\rangle$, pero se escribió:

```
from spinqit import Circuit, H
from spinqit import get_basic_simulator

circ = Circuit()
q = circ.allocateQubits(1)
circ << H(q[0])

engine = get_basic_simulator()
result = engine.execute(circ)
print(result.probabilities)
```

Identificar al menos dos problemas.

Solución: describir, compilar y ejecutar

Hay dos problemas principales:

- la forma de agregar la compuerta debe ser `circ << (H, q[0])`;
- el backend necesita el circuito compilado y una configuración.

```
from spinqit import BasicSimulatorConfig
from spinqit import Circuit, H, draw
from spinqit import get_basic_simulator, get_compiler

circ = Circuit()
q = circ.allocateQubits(1)
circ << (H, q[0])

compiler = get_compiler("native")
ir = compiler.compile(circ, 0)
draw(ir, "circuitoH.png")

engine = get_basic_simulator()
config = BasicSimulatorConfig()
config.configure_shots(1024)
config.configure_measure_qubits([0])

result = engine.execute(ir, config)
print(result.probabilities)
```

De la simulación a la SpinQ Triangulum

Del simulador al hardware real

En un simulador, las compuertas se ejecutan de manera ideal.

En un dispositivo cuántico real aparecen **ruido y decoherencia**: interacciones no deseadas con el entorno pueden alterar la información cuántica.

Los dispositivos actuales suelen describirse como **NISQ** (*Noisy Intermediate-Scale Quantum*).

Esto implica que:

- las operaciones no son perfectas;
- los circuitos demasiado largos acumulan errores;
- la calidad de los resultados depende del hardware y del entorno.

Errores de compuerta y comparación con el simulador

En el hardware, una compuerta puede apartarse ligeramente de la transformación ideal:

- los pulsos de control pueden ser imperfectos;
- las rotaciones pueden desviarse del ángulo previsto;
- los errores pueden acumularse durante el circuito.

Por eso, la distribución medida puede diferir de la predicción teórica.

> Conviene conservar siempre una **línea de base simulada** para comparar el comportamiento ideal con el resultado obtenido en el dispositivo físico.

Dos formas de ejecutar el mismo circuito

El circuito puede mantenerse igual mientras cambia el backend:

Backend	Propósito	Información principal
`get_basic_simulator()`	probar y depurar	estado, probabilidades y conteos
`get_nmr()`	ejecutar en la Triangulum	resultados experimentales, principalmente probabilidades

Primero ejecutaremos en el simulador. Después podremos comparar con el dispositivo físico.

Diferencias con el backend NMR

En la ejecución física:

- no podemos inspeccionar directamente el vector `states`;
- el equipo realiza un experimento físico y devuelve probabilidades experimentales;
- los resultados pueden diferir de los valores ideales por preparación, control, decoherencia y lectura.

> El simulador calcula una evolución ideal; la Triangulum realiza un experimento físico.

Conexión con la Triangulum

El siguiente código es una plantilla para ejecutar nuestro circuito en la máquina real.

```
from spinqit import Circuit, H, NMRConfig, draw
from spinqit import get_compiler, get_nmr

circ = Circuit()
q = circ.allocateQubits(1)
circ << (H, q[0])

compiler = get_compiler("native")
ir = compiler.compile(circ, 0)
draw(ir, "triangulum_H.png")

...
```

Conexión con la Triangulum (Continuación)

Los datos de `ip`, `port` y `account` deben reemplazarse por la configuración correspondiente.

```
...

engine = get_nmr()
config = NMRConfig()
config.configure_shots(1024)
config.configure_ip("IP_DE_TRIANGULUM")
config.configure_port(55444)
config.configure_account("USUARIO", "CONTRASENA_PRIVADA")
config.configure_task("clase4_H", "Superposicion de un qubit")

result = engine.execute(ir, config)
print("probabilities:", result.probabilities)
```

¿Qué configura cada método?

Método	Función
`NMRConfig()`	crea la configuración para el backend NMR
`configure_shots`	indica cuántas repeticiones solicitar
`configure_ip`	indica la dirección del equipo
`configure_port`	indica el puerto de comunicación
`configure_account`	establece usuario y contraseña
`configure_task`	agrega nombre y descripción de la tarea

La configuración de red y de cuenta debe seguir las indicaciones del responsable del equipo.

Resumen: Ideas fundamentales de esta clase

- Un operador cuántico es **lineal** y su matriz se construye a partir de su acción sobre una base.
- Las compuertas cuánticas se representan mediante operadores **unitarios**.
- SpinQit permite describir, compilar y ejecutar circuitos.
- Las amplitudes contienen **módulo y fase**; la interferencia surge al combinarlas.
- El simulador predice el comportamiento ideal; el hardware realiza un **experimento físico**.

Apéndice A - Material optativo

Deducción de la Esfera de Bloch

¿De dónde surge la esfera de Bloch?

Un estado puro de un qubit puede escribirse como

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

donde α y β son números complejos:

$$\alpha = a + bi, \qquad \beta = c + di.$$

Por lo tanto, inicialmente necesitamos **cuatro números reales**:

$$a,\ b,\ c,\ d.$$

Sin embargo, la normalización y la fase global reducirán esa cantidad a solo **dos parámetros reales**.

> Nuestro objetivo es llegar a los ángulos θ y φ de la esfera de Bloch.

Paso 1: aplicar la normalización

Para representar un estado físico mediante un ket normalizado debe cumplirse:

$$|\alpha|^2 + |\beta|^2 = 1.$$

Escribamos las amplitudes en forma polar:

$$\alpha = |\alpha| e^{i\varphi_\alpha}, \qquad \beta = |\beta| e^{i\varphi_\beta}.$$

Ahora tenemos los módulos y las fases:

$$|\alpha|, \quad |\beta|, \quad \varphi_\alpha, \quad \varphi_\beta.$$

Pero los módulos no son independientes:

$$|\alpha|^2 + |\beta|^2 = 1.$$

> La normalización reduce los **cuatro parámetros reales iniciales a tres parámetros independientes**.

Paso 2: eliminar la fase global

Partimos de

$$|\psi\rangle = |\alpha|e^{i\varphi_\alpha}|0\rangle + |\beta|e^{i\varphi_\beta}|1\rangle.$$

Multiplicamos todo el ket por $e^{-i\varphi_\alpha}$:

$$|\psi'\rangle = e^{-i\varphi_\alpha}|\psi\rangle.$$

Como solo hemos cambiado la **fase global**, $|\psi\rangle$ y $|\psi'\rangle$ pertenecen al mismo rayo y representan el mismo estado físico.

Entonces:

$$|\psi'\rangle = |\alpha||0\rangle + |\beta|e^{i(\varphi_\beta-\varphi_\alpha)}|1\rangle.$$

Definimos la **fase relativa**

$$\varphi = \varphi_\beta - \varphi_\alpha.$$

¿Qué conseguimos al eliminar la fase global?

Podemos representar el mismo estado físico mediante

$$|\psi\rangle = |\alpha||0\rangle + |\beta|e^{i\varphi}|1\rangle.$$

Ahora solo aparecen:

$$|\alpha|, \qquad |\beta|, \qquad \varphi.$$

Pero todavía tenemos la condición

$$|\alpha|^2 + |\beta|^2 = 1.$$

Por lo tanto, $|\alpha|$ y $|\beta|$ pueden describirse mediante **un único parámetro**.

> El estado físico de un qubit queda determinado por solo **dos parámetros reales**: uno para los módulos y otro para la fase relativa.

Paso 3: introducir el ángulo θ

Como

$$|\alpha|^2 + |\beta|^2 = 1$$

y ambos módulos son números reales no negativos, podemos parametrizarlos mediante seno y coseno:

$$|\alpha| = \cos\left(\frac{\theta}{2}\right), \qquad |\beta| = \sin\left(\frac{\theta}{2}\right).$$

La normalización queda automáticamente satisfecha:

$$\cos^2\left(\frac{\theta}{2}\right) + \sin^2\left(\frac{\theta}{2}\right) = 1.$$

Como $|\alpha|, |\beta| \geq 0$:

$$0 \leq \frac{\theta}{2} \leq \frac{\pi}{2} \qquad \Longrightarrow \qquad 0 \leq \theta \leq \pi.$$

La representación general de un qubit

Sustituyendo los módulos obtenemos:

$$\boxed{|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\theta}{2}\right)|1\rangle}$$

con

$$0 \leq \theta \leq \pi, \qquad 0 \leq \varphi < 2\pi.$$

Hemos pasado de cuatro números reales

$$a,\ b,\ c,\ d$$

a solo dos ángulos:

$$\theta, \qquad \varphi.$$

La normalización elimina un grado de libertad y la fase global elimina otro.

De los dos ángulos a la esfera

Los ángulos θ y φ pueden interpretarse como las coordenadas esféricas de un punto de una esfera de radio 1:

$$x = \sin\theta\cos\varphi,$$
$$y = \sin\theta\sin\varphi,$$
$$z = \cos\theta.$$

Por construcción:

$$x^2 + y^2 + z^2 = 1.$$

Por eso cada estado físico puro de un qubit puede representarse mediante un punto de la **esfera de Bloch**.

$$|\psi\rangle \quad \longleftrightarrow \quad (x, y, z).$$

Algunos estados conocidos

Para $\theta = 0$:

$$|\psi\rangle = |0\rangle \qquad \longleftrightarrow \qquad (0, 0, 1).$$

Para $\theta = \pi$:

$$|\psi\rangle = |1\rangle \qquad \longleftrightarrow \qquad (0, 0, -1).$$

En el ecuador, $\theta = \frac{\pi}{2}$:

$$|\psi\rangle = \frac{|0\rangle + e^{i\varphi}|1\rangle}{\sqrt{2}}.$$

Por ejemplo:

$$\varphi = 0 \Rightarrow |+\rangle, \qquad \varphi = \pi \Rightarrow |-\rangle.$$

Los estados ortogonales aparecen en puntos antipodales de la esfera de Bloch.

Apéndice B - Material optativo

Pensar como programadores cuánticos

Separar cinco preguntas

Ante cualquier circuito conviene responder:

1. **¿Cuál es el estado inicial?** Por defecto, $|0\rangle$.
2. **¿Qué operaciones aplico y en qué orden?** El circuito define la evolución del estado.
3. **¿Qué estado final predice la matemática?** Se obtiene aplicando los operadores correspondientes.
4. **¿Qué voy a medir?** Debemos decidir qué qubits medir y en qué base.
5. **¿Qué devuelve el backend?** Según el backend y la configuración, podemos obtener probabilidades, estados o resultados experimentales.

> Separar estas preguntas ayuda a distinguir la evolución cuántica, la medición y la implementación del programa.

Una estrategia para depurar circuitos

Cuando un resultado no coincide con lo esperado, conviene revisar en este orden:

1. **Estado inicial:** ¿es el que esperamos?
2. **Circuito:** ¿las compuertas y su orden son correctos?
3. **Matemática:** ¿qué estado debería obtenerse idealmente?
4. **Compilación:** ¿estamos ejecutando el circuito compilado?
5. **Medición:** ¿medimos los qubits y la base correctos?
6. **Backend:** ¿estamos usando simulador o hardware?

> Primero comprobamos la matemática; después buscamos el problema en el programa o en el dispositivo.

Una función reutilizable

Cuando queremos comparar circuitos, conviene encapsular el código repetido:

```
from spinqit import BasicSimulatorConfig
from spinqit import get_basic_simulator, get_compiler


def ejecutar(circ, shots=1024):
    compiler = get_compiler("native")
    ir = compiler.compile(circ, 0)

    engine = get_basic_simulator()
    config = BasicSimulatorConfig()
    config.configure_shots(shots)
    config.configure_measure_qubits([0])

    result = engine.execute(ir, config)
    return result
```

La función recibe un circuito de un qubit y devuelve su resultado.

Construir circuitos a partir de una lista

Podemos describir una secuencia de compuertas como una lista:

```
from spinqit import Circuit


def construir_circuito(puertas):
    circ = Circuit()
    q = circ.allocateQubits(1)
    for puerta in puertas:
        circ << (puerta, q[0])
    return circ
```

Por ejemplo:

```
from spinqit import H, X

circ = construir_circuito([H, X, H])
result = ejecutar(circ)
print(result.probabilities)
```

Apéndice C - Material optativo

El producto externo

El producto externo

Dados dos kets $|v\rangle$ y $|w\rangle$, el **producto externo**

$$|v\rangle\langle w|$$

define un **operador lineal**.

Podemos escribir:

$$\hat{A} = |v\rangle\langle w|.$$

A diferencia del producto interno, $\langle w|v\rangle \in \mathbb{C}$, que produce un escalar, el producto externo define una transformación que puede actuar sobre otros vectores.

> El producto externo es una herramienta para **construir y describir operadores** sin necesidad de comenzar por una matriz.

¿Cómo actúa $|v\rangle\langle w|$?

Sea $\hat{A} = |v\rangle\langle w|$. Al aplicarlo sobre un ket arbitrario $|\psi\rangle$:

$$\hat{A}|\psi\rangle = |v\rangle\langle w|\psi\rangle = \langle w|\psi\rangle\,|v\rangle.$$

El producto interno $\langle w|\psi\rangle$ produce un escalar complejo que determina cuánto se escala el vector $|v\rangle$. Por lo tanto:

$$\boxed{|v\rangle\langle w| : \ |\psi\rangle \longmapsto \langle w|\psi\rangle\,|v\rangle}$$

> Primero obtenemos el solapamiento con $|w\rangle$; luego ese número multiplica a $|v\rangle$.

Del operador abstracto a su matriz

Sea una base ortonormal $\mathcal{B} = (|e_1\rangle, \ldots, |e_n\rangle)$. La matriz que representa al operador $\hat{A}$ en esa base se denota como $[\hat{A}]_{\mathcal{B}}$.

Sea $A = [\hat{A}]_{\mathcal{B}}$, A tiene elementos

$$\boxed{A_{ij} = \langle e_i | \hat{A} | e_j \rangle}$$

Para un producto externo $\hat{A} = |v\rangle\langle w|$, obtenemos en coordenadas:

$$[\hat{A}]_{\mathcal{B}} = [|v\rangle]_{\mathcal{B}}[\langle w|]_{\mathcal{B}}.$$

> El operador $|v\rangle\langle w|$ es abstracto; la matriz aparece cuando elegimos una base.

Ejemplo: construir el operador $\hat{Z}$

Podemos definir abstractamente: $\hat{Z} = |0\rangle\langle 0| - |1\rangle\langle 1|$.

Su acción sobre la base computacional es:

$$\hat{Z}|0\rangle = |0\rangle, \qquad \hat{Z}|1\rangle = -|1\rangle.$$

En la base $\mathcal{B}_Z = (|0\rangle, |1\rangle)$, los dos productos externos se representan como

$$|0\rangle\langle 0| \longrightarrow \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad |1\rangle\langle 1| \longrightarrow \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Por lo tanto:

$$[\hat{Z}]_{\mathcal{B}_Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = Z.$$

Ejemplo: construir el operador $\hat{Z}$

Vamos a aplicar la "receta" para construir la matriz del operador $\hat{Z}$ sobre la base computacional, sabiendo que $\hat{Z}|0\rangle = |0\rangle$ y $\hat{Z}|1\rangle = -|1\rangle$

1. Calculamos la primera columna: $\hat{Z}|0\rangle = |0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

2. Calculamos la segunda columna: $\hat{Z}|1\rangle = -|1\rangle = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$

3. Ensamblamos la matriz:

$$Z = \begin{pmatrix} \hat{Z}|0\rangle & \hat{Z}|1\rangle \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

El mismo operador en otra base

El operador $\hat{Z}$ sigue siendo el mismo, pero también podemos describir su acción utilizando los kets $|+\rangle$ y $|-\rangle$ que represnetan la base $\mathcal{B}_X = \{|+\rangle, |-\rangle\}$.

Como $\hat{Z}|+\rangle = |-\rangle \quad \text{y} \quad \hat{Z}|-\rangle = |+\rangle$, en esta base, las coordenadas de las imágenes son

$$[\hat{Z}|+\rangle]_{\mathcal{B}_X} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad [\hat{Z}|-\rangle]_{\mathcal{B}_X} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Por lo tanto:

$$[\hat{Z}]_{\mathcal{B}_X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

> El operador $\hat{Z}$ no cambió: cambió su **representación matricial** porque elegimos otra base.